

Informe de Auditoría de Seguridad - KogniTerm

Proyecto: KogniTerm

Alcance: Código fuente, sistema de skills, gestión de memoria, configuración y dependencias

Estandares: OWASP Top 10 / ASVS, Secure Coding Practices

Auditoría ejecutada con: Skill security-auditor (metodología de 7 pasos)

Fecha: 2025-01-27

Estado: Paso 6 completado - Informe final

Resumen Ejecutivo

KogniTerm es un agente de terminal AI con backend FastAPI, frontend TUI basado en Rich, orquestación de agentes con LangGraph, sistema de skills dinámicas y soporte para múltiples proveedores LLM (OpenRouter, Gemini, Ollama).

La auditoría de seguridad ha identificado **13 hallazgos** clasificados por severidad: - **5 Críticos** - **4 Altos** - **3 Medios** - **1 Bajo**

Los hallazgos más críticos giran en torno a la **ausencia total de autenticación/autorización** en la API, **ejecución de comandos sin validación**, **exposición de secrets en almacenamiento plano** y **configuración CORS insegura**.

1. Metodología

Se aplicó la metodología de 7 pasos de la skill security-auditor:

1. Confirmar alcance y activos
2. Revisar arquitectura y controles existentes (base: auditoria_arquitectura.md, puntuación 7.2/10)
3. Mapear entry points y trazar flujos de datos hasta almacenamiento
4. Analizar fallos lógicos y vectores de ataque por feature
5. Verificar controles de autenticación/autorización y detectar secrets expuestos

- 6. Clasificar hallazgos por severidad y elaborar informe final
- 7. Entregar plan de remediacion (este documento incluye recomendaciones)

Herramientas utilizadas: - Revision manual de codigo fuente - bandit (analisis estatico): 236 low, 14 medium, 2 high - grep de patrones de secrets - Analisis de arquitectura previa

2. Arquitectura y Activos

Activos principales

Activo	Descripcion	Sensibilidad
API Keys LLM	Claves de OpenRouter, Google, OpenAI, Anthropic, Ollama Cloud	Critica
Tokens Telegram	Bot tokens para integracion con Telegram	Alta
Historial de chat	.kogniterm/history.json por sesion	Media
Configuracion	config.json global y por proyecto	Alta
Codigo fuente	Logica de ejecucion de comandos, skills, agentes	Critica
Sesiones	Estado de agentes en memoria + disco	Media

Entry Points identificados

Tipo	Ruta	Funcion
REST	POST /api/execute	Ejecucion de comandos shell
REST	POST /api/files/list	Listado de directorios
REST	POST /api/config/llm	Modificacion de configuracion LLM y API keys
REST	POST /api/config/set	Escritura arbitraria de config
REST		

Tipo	Ruta	Funcion
	GET /api/config/all	Lectura de configuracion completa
REST	POST /config/channels	Gestion de canales (incluye tokens)
REST	DELETE /config/channels/{name}	Eliminacion de canales
REST	PATCH /config/channels/{name}/toggle	Activacion/desactivacion
REST	GET /health	Informacion de sesiones y canales
REST	GET /sessions	Listado de sesiones
REST	POST /sessions	Creacion de sesiones
REST	DELETE /sessions/{session_id}	Eliminacion de sesiones
REST	POST /api/chat/message	Chat sincrono
REST	POST /chat/{session_id}	Chat por sesion
REST	GET /sse/{session_id}	SSE streaming
WebSocket	/ws/chat	Sesion desktop unica
WebSocket	/ws/{session_id}	Sesion persistente
WebSocket	terminal_input	Input directo a PTY
WebSocket	start_indexing	Indexacion de codebase
WebSocket	approval_response	Respuestas de aprobacion

3. Hallazgos por Severidad

3.1 Hallazgos Criticos

AUTH-01: Sin autenticacion/autorizacion en la API

Severidad: Critica

CWE: CWE-306 (Missing Authentication for Critical Function)

OWASP: A01:2025 - Broken Access Control

Ubicacion: server/app.py - todos los endpoints

Descripcion:

Ningun endpoint de la API requiere autenticacion, autorizacion o verificacion de identidad. Cualquier cliente que pueda alcanzar el servidor puede ejecutar comandos, leer/escribir configuracion, gestionar sesiones y acceder a datos sensibles.

Evidencia:

```
# server/app.py - ejemplo de endpoint sin proteccion
@app.post("/api/execute")
async def execute_command(request: CommandRequest):
    session = pool.get_or_create("default")
    # ... ejecucion directa sin verificar identidad
```

Impacto: - Acceso total a funcionalidades administrativas y operativas - Ejecucion remota de codigo (RCE) via endpoint de comandos - Exfiltracion de API keys y tokens - Manipulacion de configuraciones y sesiones

Recomendacion: 1. Implementar middleware de autenticacion (API key, JWT, o similar) 2. Agregar verificacion de autorizacion por rol/usuario 3. Proteger especialmente endpoints sensibles: /api/execute, /api/config/*, /config/channels 4. Considerar OAuth2 o mTLS para entornos multi-usuario

CMD-01: Ejecucion de comandos sin validacion

Severidad: Critica

CWE: CWE-78 (OS Command Injection)

OWASP: A03:2025 - Injection

Ubicacion: server/app.py:847-876, core/command_executor.py:153-366

Descripcion:

El endpoint POST /api/execute y el WebSocket terminal_input permiten enviar comandos shell arbitrarios que se ejecutan en el servidor sin validacion de contenido, filtrado de caracteres peligrosos o restriccion de privilegios.

Evidencia:

```
# server/app.py
@app.post("/api/execute")
async def execute_command(request: CommandRequest):
    session = pool.get_or_create("default")
    # El comando se ejecuta directamente sin validacion
    for chunk in
        session.command_executor.execute(request.command):
        yield chunk

# core/command_executor.py
def execute(self, command: str, ...):
```

```
# El comando se envia al PTY sin validacion
os.write(master_fd, full_cmd.encode())
```

Impacto: - Ejecucion arbitraria de comandos en el servidor -
Acceso a archivos sensibles (/etc/passwd, .env, config.json) -
Posible pivoting a otros sistemas de la red - Exfiltracion de datos

Recomendacion: 1. Implementar allowlist de comandos permitidos por rol 2. Agregar filtrado de caracteres peligrosos (;, |, &, \$(), `) 3. Ejecutar comandos en contenedor/sandbox aislado (Docker, gVisor) 4. Implementar principio de minimo privilegio (usuario no-root) 5. Agregar logging detallado de todos los comandos ejecutados

PATH-01: Path Traversal en listado de archivos

Severidad: Critica

CWE: CWE-22 (Path Traversal)

OWASP: A01:2025 - Broken Access Control

Ubicacion: server/app.py:873-876

Descripcion:

El endpoint POST /api/files/list usa os.path.abspath(request.path) sin validacion de que la ruta resultante este dentro del workspace permitido. Un atacante puede especificar rutas como ../../../../etc/passwd para leer archivos fuera del workspace.

Evidencia:

```
# server/app.py
@app.post("/api/files/list")
async def list_directory(request: DirectoryRequest):
    target_path = os.path.abspath(request.path)
    # No hay validacion de que target_path este dentro del
    # workspace
    entries = os.scandir(target_path)
```

Impacto: - Lectura de archivos sensibles del sistema (/etc/passwd, /etc/shadow) - Lectura de archivos de configuracion con API keys - Exfiltracion de codigo fuente y datos de usuario

Recomendacion: 1. Validar que la ruta resultante este dentro del workspace permitido 2. Usar os.path.realpath() para resolver symlinks 3. Implementar allowlist de directorios accesibles 4. Negar acceso a rutas que contengan .. o symlinks a directorios sensibles

CONF-01: API keys y secrets en JSON plano sin cifrado

Severidad: Critica

CWE: CWE-312 (Cleartext Storage of Sensitive Information)

OWASP: A02:2025 - Cryptographic Failures

Ubicacion: terminal/config_manager.py:42-45, server/
config.py:42-45

Descripcion:

Las API keys, tokens y otros secrets se almacenan en archivos JSON planos (config.json, server_config.json) sin cifrado. Cualquier proceso con acceso al filesystem puede leer estas credenciales.

Evidencia:

```
# terminal/config_manager.py
def _save_json(self, path: Path, data: Dict[str, Any]):
    with open(path, 'w', encoding='utf-8') as f:
        json.dump(data, f, indent=4) # Sin cifrado
```

Impacto: - Exfiltracion de API keys de proveedores LLM - Robo de tokens de Telegram/Discord/Slack - Acceso a servicios externos con las credenciales del usuario - Posible facturacion fraudulenta

Recomendacion: 1. Implementar cifrado de secrets usando cryptography (Fernet, AES-GCM) 2. Usar un keyring del sistema (keyring, libsecret, Windows Credential Manager) 3. Cifrar el archivo de configuracion con una clave derivada de contraseña maestra 4. Nunca escribir secrets en logs o archivos temporales

CONF-02: Escritura de configuracion sin validacion

Severidad: Critica

CWE: CWE-94 (Improper Control of Generation of Code)

OWASP: A03:2025 - Injection

Ubicacion: server/app.py:664-680

Descripcion:

El endpoint POST /api/config/set acepta cualquier clave y valor sin validacion de tipo, formato o rango. Un atacante puede inyectar valores maliciosos que afecten el comportamiento del sistema.

Evidencia:

```
# server/app.py
@app.post("/api/config/set")
async def set_config_value(req: SetConfigRequest = Body(...)):
    cm.set_global_config(req.key, req.value)
    # No hay validacion de req.key ni req.value
```

Impacto: - Modificación de configuración crítica (modelos LLM, endpoints) - Posible inyección de configuraciones maliciosas - Denegación de servicio por configuraciones inválidas

Recomendación: 1. Implementar allowlist de claves configurables 2. Validar tipos y formatos de valores 3. Agregar verificación de firma para cambios críticos 4. Implementar auditoría de cambios de configuración

3.2 Hallazgos Altos

SSRF-01: SSRF en endpoint de modelos disponibles

Severidad: Alta

CWE: CWE-918 (Server-Side Request Forgery)

OWASP: A10:2025 - Server-Side Request Forgery

Ubicación: server/app.py:304-498

Descripción:

El endpoint GET /api/models/available realiza peticiones a APIs externas (OpenRouter, Google, etc.) usando las API keys del sistema. Un atacante podría manipular parámetros para acceder a recursos internos o exfiltrar credenciales.

Evidencia:

```
# server/app.py - líneas 304-498
# El endpoint hace peticiones a URLs externas sin validación
# usando las API keys del sistema
```

Impacto: - Acceso a metadatos de red interna - Exfiltración de API keys en URLs de petición - Acceso a servicios internos no expuestos públicamente

Recomendación: 1. Implementar allowlist de dominios permitidos 2. No incluir API keys en URLs (usar headers) 3. Validar y sanitizar todas las URLs de destino 4. Considerar un proxy inverso para peticiones externas

CORS-01: CORS completamente abierto

Severidad: Alta

CWE: CWE-942 (Overly Permissive Cross-domain Whitelist)

OWASP: A05:2025 - Security Misconfiguration

Ubicación: server/app.py:206-212

Descripcion:

La configuracion CORS permite cualquier origen (*) con credenciales (allow_credentials=True), lo que permite a cualquier dominio web realizar peticiones autenticadas al servidor.

Evidencia:

```
# server/app.py
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Permite cualquier origen
    allow_credentials=True, # Con credenciales
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Impacto: - CSRF (Cross-Site Request Forgery) - Robo de sesiones y credenciales - Ejecucion de acciones no autorizadas desde sitios maliciosos

Recomendacion: 1. Especificar dominios permitidos explicitamente en allow_origins 2. Deshabilitar allow_credentials si no es estrictamente necesario 3. Limitar metodos y headers a los estrictamente necesarios 4. Considerar usar tokens CSRF para operaciones sensibles

WS-01: Manipulacion de workspace_dir en WebSocket

Severidad: Alta

CWE: CWE-22 (Path Traversal)

OWASP: A01:2025 - Broken Access Control

Ubicacion: server/app.py:1164-1170, server/session_pool.py:542

Descripcion:

El WebSocket /ws/{session_id} acepta un parametro workspace_dir opcional que se usa para establecer el directorio de trabajo de la sesion sin validacion. Un atacante puede especificar rutas arbitrarias para acceder a archivos fuera del workspace esperado.

Evidencia:

```
# server/app.py
@app.websocket("/ws/{session_id}")
async def websocket_chat(websocket: WebSocket, session_id: str,
    workspace_dir: Optional[str] = None):
    session = pool.get_or_create(session_id,
        workspace_dir=workspace_dir)
    # workspace_dir se usa sin validacion
```


Impacto: - Acceso a archivos fuera del workspace permitido - Lectura de archivos sensibles del sistema - Modificación de archivos en directorios no autorizados

Recomendacion: 1. Validar que workspace_dir este dentro de rutas permitidas 2. Implementar allowlist de workspaces accesibles por sesion 3. Usar os.path.realpath() para resolver symlinks 4. Negar acceso a rutas que contengan ..

RACE-01: Race condition en SessionPool.get_or_create()

Severidad: Alta

CWE: CWE-362 (Race Condition)

OWASP: A04:2025 - Insecure Design

Ubicacion: server/session_pool.py:1086-1118

Descripcion:

El metodo get_or_create() no es thread-safe. Multiples requests concurrentes pueden crear sesiones duplicadas o acceder a estado inconsistente, permitiendo potencialmente bypassear logica de sesiones.

Evidencia:

```
# server/session_pool.py
def get_or_create(self, session_id: str, workspace_dir:
    Optional[str] = None) -> AgentSession:
    if session_id in self.sessions:
        return self.sessions[session_id]
    # Race condition: multiples threads pueden crear la misma
    # sesion
    session = AgentSession(...)
    self.sessions[session_id] = session
    return session
```

Impacto: - Creacion de sesiones duplicadas - Estado inconsistente de sesiones - Potencial bypass de controles de sesion

Recomendacion: 1. Usar threading.Lock() o asyncio.Lock() para proteger la seccion critica 2. Implementar patron double-checked locking 3. Considerar usar un diccionario thread-safe (collections.defaultdict con lock)

3.3 Hallazgos Medios

INFO-01: Health check expone informacion sensible

Severidad: Media

CWE: CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor)

OWASP: A01:2025 - Broken Access Control

Ubicacion: server/app.py:217-235

Descripcion:

El endpoint GET /health devuelve informacion detallada sobre sesiones activas, configuracion de canales y estado del sistema, lo que facilita la fase de reconocimiento de un atacante.

Evidencia:

```
# server/app.py
@app.get("/health")
async def health():
    sessions = pool.list_all()
    channels = server_config.settings.channels
    return {
        "status": "ok",
        "sessions": [s.session_id for s in sessions],
        "channels": [c.name for c in channels if c.enabled],
        # ... mas informacion sensible
    }
```

Impacto: - Revela estructura interna del sistema - Enumera sesiones activas para targeting - Expone configuracion de canales

Recomendacion: 1. Limitar informacion devuelta a estado basico (up/down) 2. Requerir autentificacion para detalles adicionales 3. No exponer nombres de sesiones o configuracion interna

SECRET-01: Tokens y secrets hardcodeados

Severidad: Media

CWE: CWE-798 (Use of Hard-coded Credentials)

OWASP: A07:2025 - Identification and Authentication Failures

Ubicacion: core/llm_service.py:214, server/config.py:22

Descripcion:

Se detectaron secrets hardcodeados en el codigo fuente: -
llm_service.py:214: self.api_key = "antigravity-session-token" -
server_config.py:22: "YOUR_TELEGRAM_BOT_TOKEN" como valor default

Evidencia:

```
# core/llm_service.py
if self.model_name.startswith("antigravity/"):
    self.api_key = "antigravity-session-token" # Hardcodeado

# server/config.py
ChannelConfig(
    name="telegram_bot_default",
    type="telegram_bot",
    enabled=False,
    params={"token": "YOUR_TELEGRAM_BOT_TOKEN"} # Placeholder
    hardcodedo
)
```

Impacto: - Credenciales expuestas en el repositorio de código - Dificultad para rotar secrets - Posible uso de secrets de prueba en producción

Recomendación: 1. Mover todos los secrets a variables de entorno o archivos de configuración externos 2. Usar gestores de secretos (HashiCorp Vault, AWS Secrets Manager) 3. Implementar rotación automática de credenciales 4. Escanear repositorios regularmente en busca de secrets

BANDIT-01: Generador no criptográfico (B311)

Severidad: Media

CWE: CWE-330 (Use of Insufficiently Random Values)

OWASP: A02:2025 - Cryptographic Failures

Ubicación: ui/visual_components.py:692

Descripción:

Se detectó el uso de random (no criptográfico) en un contexto que podría requerir valores aleatorios seguros.

Evidencia:

```
# ui/visual_components.py:692
# Uso de random para generacion de valores que podrian ser
sensibles
```

Impacto: - Valores predecibles en contextos sensibles - Potencial explotación si se usa para tokens, IDs, etc.

Recomendación: 1. Reemplazar random por secrets para valores criptográficos 2. Usar os.urandom() o secrets.token_bytes() para bytes aleatorios 3. Auditar todos los usos de random en el código

3.4 Hallazgos Bajos

BANDIT-02: try/except pass (B110)

Severidad: Baja
CWE: CWE-703 (Improper Check of Special Conditions)
OWASP: A04:2025 - Insecure Design
Ubicacion: Multiples archivos

Descripcion:
Multiples bloques try/except con pass silencian errores sin logging ni manejo apropiado, dificultando la deteccion de problemas.

Evidencia:

```
# Ejemplo en command_executor.py
try:
    # ... operacion
except:
    pass # Error silenciado
```

Impacto: - Errores ocultos que dificultan el debugging - Posible fallo silencioso de controles de seguridad - Perdida de informacion de auditoria

Recomendacion: 1. Reemplazar except: pass por except Exception as e: logger.error(...) 2. Definir politicas de manejo de errores especificas por tipo 3. No capturar excepciones de forma generica sin logging

4. Resumen de Hallazgos

ID	Hallazgo	Severidad	CWE	OWASP	Archivo
AUTH-01	Sin autenticacion en API	Critica	CWE-306	A01:2025	server/app.py
CMD-01	Ejecucion de comandos sin validacion	Critica	CWE-78	A03:2025	server/app.py, command_executor.py
PATH-01	Path traversal en listado de archivos	Critica	CWE-22	A01:2025	server/app.py
CONF-01	API keys en JSON plano sin cifrado	Critica	CWE-312	A02:2025	terminal/config_manager.py, server/config.py
CONF-02		Critica	CWE-94	A03:2025	server/app.py

ID	Hallazgo	Severidad	CWE	OWASP	Archivo
	Escritura de configuracion sin validacion				
SSRF-01	SSRF en endpoint de modelos	Alta	CWE-918	A10:2025	server/app.py
CORS-01	CORS completamente abierto	Alta	CWE-942	A05:2025	server/app.py
WS-01	Manipulacion de workspace_dir	Alta	CWE-22	A01:2025	server/app.py, server/ session_pool.py
RACE-01	Race condition en sesiones	Alta	CWE-362	A04:2025	server/ session_pool.py
INFO-01	Health check expone informacion	Media	CWE-200	A01:2025	server/app.py
SECRET-01	Tokens hardcodeados	Media	CWE-798	A07:2025	core/llm_service/ server/config.py
BANDIT-01	Generador no criptografico	Media	CWE-330	A02:2025	ui/ visual_componen
BANDIT-02	try/except pass	Baja	CWE-703	A04:2025	Multiples arch.

5. Recomendaciones Priorizadas

Prioridad 1 (Critica) - Implementar inmediatamente

- 1. Implementar autenticacion en la API**
 - Agregar middleware de autenticacion (API key, JWT)
 - Proteger todos los endpoints sensibles
 - Implementar autorizacion por rol
- 2. Agregar validacion de comandos**
 - Implementar allowlist de comandos permitidos
 - Agregar filtrado de caracteres peligrosos
 - Ejecutar en sandbox aislado
- 3. Corregir path traversal**
 - Validar rutas contra workspace permitido
 - Usar `os.path.realpath()` para resolver symlinks
 - Implementar allowlist de directorios
- 4. Cifrar secrets en almacenamiento**
 - Implementar cifrado de `config.json`
 - Usar keyring del sistema

- Nunca escribir secrets en logs

Prioridad 2 (Alta) - Implementar en corto plazo

1. Corregir CORS

- Especificar dominios permitidos
- Revisar necesidad de `allow_credentials`

2. Validar `workspace_dir`

- Agregar validación en WebSocket
- Implementar allowlist de workspaces

3. Corregir race conditions

- Agregar locks en SessionPool
- Revisar concurrencia en sesiones

4. Implementar rate limiting

- Aplicar rate limiting en endpoints sensibles
- Configurar limites por usuario/IP

Prioridad 3 (Media) - Mejora continua

1. Reducir informacion en health check

- Limitar a estado basico
- Requerir auth para detalles

2. Eliminar secrets hardcodedos

- Mover a variables de entorno
- Implementar rotacion

3. Mejorar manejo de errores

- Reemplazar `try/except pass` por logging apropiado
-

6. Conclusiones

KogniTerm presenta **vulnerabilidades criticas** que deben ser addressed antes de exponer el servicio a redes no confiables o usuarios no autenticados. Los hallazgos mas criticos giran en torno a:

1. Ausencia total de controles de acceso (AUTH-01)

2. Ejecucion arbitraria de comandos (CMD-01)

3. Exposicion de credenciales en almacenamiento plano (CONF-01)

Recomendacion principal: No exponer la API de KogniTerm a redes publicas o usuarios no confiables hasta implementar al menos: - Autenticacion obligatoria en todos los endpoints - Validacion y sandboxing de comandos - Cifrado de secrets en almacenamiento

7. Anexos

A. Herramientas utilizadas

- bandit - Analisis estatico de seguridad
- grep - Busqueda de patrones de secrets
- Revision manual de codigo
- Auditoria de arquitectura previa (auditoria_arquitectura.md)

B. Referencias

- OWASP Top 10 (2025)
- OWASP ASVS (Application Security Verification Standard)
- CWE Top 25
- Secure Coding Practices (OWASP)

C. Contacto

Para consultas sobre este informe, contactar al equipo de seguridad.

Informe generado automaticamente por el agente de auditoria de seguridad.
Skill utilizada: security-auditor